

SDR KT Onboarding Agent — Full Project Summary

How every feature works, how the numbers are calculated, and how it was tested

Anjali Roy

Shreya Anekar

2026-06-04

Table of contents

1	What this project is (in one paragraph)	2
2	The one pattern behind everything: the envelope	2
2.1	How a skill produces its answer	2
2.2	How the pieces fit together	3
3	The workflow at a glance	3
4	Feature by feature — what it does, the flow, and the code	3
4.1	Pre-boarding chain — Step 1: Generate onboarding plan	3
4.2	Pre-boarding chain — Step 2: Rank Trail Guide mentors	4
4.3	Pre-boarding chain — Step 3: Draft welcome email	4
4.4	How the chain actually advances (and pauses)	4
4.5	AI Tutor (answer-sdr-question)	5
4.6	Cold-Call Simulation (simulate-cold-call + debrief-cold-call)	5
4.7	Coaching Notes (score-cold-call)	5
4.8	Certification Gap Analysis (score-certification)	6
4.9	Onboarding Debrief (generate-handoff-doc)	6
5	How every score and threshold is calculated	6
5.1	RAG retrieval — two retrievers, two different “weak” rules	7
5.2	Confidence thresholds — the complete table	7
5.3	Cold-call / coaching score — the weighted average	7
5.4	Certification — banding, not averaging	8
5.5	Mentor ranking — see Section 6.2	8
6	What we are doing better, and how the numbers are derived	8
6.1	Time saved per step	8
6.2	How the cost numbers are calculated	8
6.3	The four quality improvements (not just time)	9
7	How we tested it	9
7.1	Layer 1 — the offline pytest suite (153 tests)	9
7.2	Layer 2 — the DeepEval quality suite (the milestone’s testing deliverable)	9
7.3	The before/after experiment (we changed one thing and measured)	10
8	Failures and limitations we actually found	10
9	The safety net: humans stay in control	11
10	One-line summary	11

1 What this project is (in one paragraph)

We built an **AI coworker** that walks a new Salesforce SDR (Sales Development Representative) through their full 12-week ramp: it drafts a personalised onboarding plan before Day 1, answers product/policy questions with cited sources, role-plays prospects for cold-call practice, turns real call transcripts into coaching notes, scores the SDR at the certification gate against a rubric with quoted evidence, and — after they pass — collects a structured debrief on what worked in the program. **Every AI output is gated by a human** (approve / edit / reject / escalate), and **every decision is written to an audit log**. Two roles run through the app: the **SDR** (the trainee) and the **Manager** (the reviewer).

The whole thing runs on one repeating pattern, so once you understand that pattern you understand the whole codebase. This document explains that pattern, then walks each feature step by step, then explains exactly how every score and threshold is calculated, what we improved, and how we tested it.

2 The one pattern behind everything: the envelope

Every unit of AI work in the system is a small standalone Python script called a **skill** (LLM-powered) or an **automation** (deterministic, no LLM). They never talk to each other directly. Instead, each one prints a single JSON object — the **envelope** — on standard output, and the orchestrator reads that envelope to decide what happens next.

The envelope always has exactly **7 keys** (defined in `utils/sdr_common.py:10`):

Key	Meaning
<code>status</code>	<code>ok</code> , <code>needs_review</code> , or <code>error</code>
<code>next_action</code>	the name of the next step, or <code>done</code>
<code>confidence</code>	the model's 0–1 self-reported confidence
<code>review_required</code>	<code>true</code> → a human must look before anything proceeds
<code>artifact_refs</code>	filenames of any documents the step wrote (e.g. the plan)
<code>outputs</code>	the structured result (scores, sources, text)
<code>error</code>	populated only when <code>status == "error"</code>

This means the entire workflow is just *“run a step, read its envelope, and either advance, pause for a human, or stop on error.”* Two helpers in `utils/sdr_common.py` do the heavy lifting:

- `make_envelope(...)` builds the 7-key dict and rejects any invalid `status`.
- `needs_human_review(confidence, threshold=0.60)` returns `True` when confidence is missing or below the threshold. This is the single function that decides whether a human gate fires. The default threshold is 0.60, but each skill passes its own number (0.65 or 0.70 — see the consolidated table in Section 8).

2.1 How a skill produces its answer

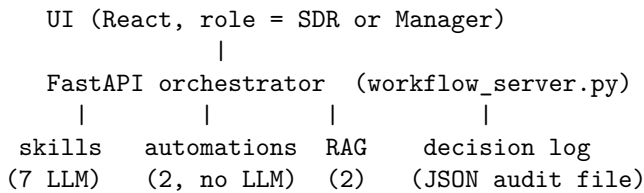
Every LLM skill follows the same five steps:

1. **Load inputs** (the hire profile, a transcript, the SDR's answers).
2. **Retrieve supporting documents** from the knowledge base (RAG — explained in Section 7).
3. **Build a prompt** = a fixed “system” instruction + the retrieved passages + the user's data.
4. **Call the LLM** through `utils/connect.py's ask_json(...)`, which forces the model to return JSON matching a **Pydantic schema** (so the output shape is guaranteed). The model used everywhere is `claude-sonnet-4-6`.
5. **Compute review triggers**, write any artifact, and emit the envelope.

Offline / no-API-key mode. Every skill first checks the `MOCK_LLM_RESPONSE` environment variable. If it is set, the skill skips the LLM entirely and validates that string straight into its schema. This is what lets all 153

offline tests run with no network and no API key — the tests inject a realistic JSON payload and exercise the full pipeline (schema validation, score recompute, review logic, artifact writing) without ever calling the model.

2.2 How the pieces fit together



- **7 LLM skills** — generate-onboarding-plan, answer-sdr-question (Tutor), score-cold-call, simulate-cold-call, debrief-cold-call, score-certification, generate-handoff-doc (the debrief).
 - **2 automations (no LLM)** — rank-trail-guides, welcome-new-hire.
 - **2 retrievers** — BM25 keyword search and ChromaDB semantic search.
 - **2 MCP servers** — expose the data and two skills as standard Model-Context-Protocol resources/tools.
 - **1 orchestrator** — workflow_server.py, the FastAPI app the UI talks to, plus a CLI version in scripts/orchestrator.py.
-

3 The workflow at a glance

There are two halves.

Half 1 — the pre-boarding chain (runs once per new hire, before Day 1):

generate-onboarding-plan (LLM + RAG) → rank-trail-guides (deterministic) → welcome-new-hire (deterministic) → **Manager gate**

The three steps hand off through the envelope. The chain **pauses at a review gate** the moment any step reports `review_required: true`.

Half 2 — five on-demand features used across the 12-week ramp:

1. **AI Tutor** — cited Q&A.
2. **Cold-Call Simulation** — practice + scored debrief.
3. **Coaching Notes** — score a real call transcript.
4. **Certification Gap Analysis** — score the SDR at the week-12 gate.
5. **Onboarding Debrief** — post-certification program feedback.

Each on-demand feature is invoked from its own UI tab and returns an envelope with `next_action = "done"`.

4 Feature by feature — what it does, the flow, and the code

4.1 Pre-boarding chain — Step 1: Generate onboarding plan

File: skills/generate-onboarding-plan/scripts/generate_onboarding_plan.py

What it does, simply: builds a personalised week-by-week Trailhead study plan for the new hire, grounded in their profile and an allow-listed module catalog.

Flow:

1. Reads the hire's profile and their `summaries/<hire_id>.md`.
2. Runs **BM25** retrieval (top 4 chunks) with a machine-built query like `"onboarding {vertical} {experience} trailhead modules territory {territory}"`.

3. Injects the **allow-list** of valid TRAIL-XXX module IDs (parsed from `trailhead_module_catalog.md`) into the prompt. The model is told: “You *MUST* only emit `module_id` values from the *ALLOW-LIST*. Do not invent module ids.”
4. The LLM returns an `OnboardingPlan`: `summary`, `weeks[1-4]` (each with `theme`, `modules`, `activities`), `skip_rationale`, `confidence`. Plan depth is calibrated to experience (entry → 4 weeks, senior → 2 weeks).
5. **Review triggers** (`review_required` is true if *any* of):
 - a module ID is **not** on the allow-list (hallucination guard), or
 - the per-hire summary file is missing, or
 - **confidence** < 0.70.
6. Writes `01_onboarding_plan.md` and sets `next_action` = “rank-trail-guides” — this is the **only** step that hands off to another step.

4.2 Pre-boarding chain — Step 2: Rank Trail Guide mentors

File: `automations/rank-trail-guides/scripts/rank_trail_guides.py` (no LLM)

What it does: scores every candidate AE mentor against the hire on a transparent **5-component, 0–100 score** and returns a ranked table.

The exact formula (`_score_row`, lines 22–28):

Component	Points	Rule
Territory match	30	exact match → 30, else 0
Vertical match	30	exact match → 30, else 0
Satisfaction	20	(<code>satisfaction_score</code> / 5) × 20
Mentorship hours/week	10	(<code>hours</code> / 5) × 10
Prior mentees	10	<code>min(prior_mentees / 10, 1.0)</code> × 10

Total = **100 max**. Candidates are sorted by score (ties broken by AE id), and the top 5 are written to `02_trail_guides.md`. Confidence is bucketed from the top score: 70 → 1.0, 50 → 0.7, otherwise 0.4 — so this step only trips the gate ($0.4 < 0.60$) when even the best mentor scores under 50.

4.3 Pre-boarding chain — Step 3: Draft welcome email

File: `automations/welcome-new-hire/scripts/welcome_new_hire.py` (no LLM)

What it does: slot-fills a fixed **four-paragraph** email template from the hire profile and the #1 mentor (parsed out of `02_trail_guides.md`). The third paragraph is chosen from one of three experience-level variants (entry / mid / senior). It writes `03_welcome_email.md` and always returns `confidence = 1.0`, `review_required = false` — so the chain auto-completes, though the manager can still edit the email before it is sent.

4.4 How the chain actually advances (and pauses)

File: `workflow_server.py`, function `_advance` (line 309).

The orchestrator runs each step as a subprocess, reads the last line of its stdout as the envelope, then:

- if `status == "error"` → mark the run **rejected** and stop;
- if `review_required` is true → **pause**: set status `paused`, store the envelope, and wait for the manager;
- otherwise → advance to the next step; if there is none, mark **complete**.

When the manager approves at the gate (`approve_run`, line 384), an **edit** overwrites the stored artifact on disk so every downstream step sees the human-corrected version, the decision is logged, and the chain resumes. Reject → status `rejected`; escalate → status `escalated`.

4.5 AI Tutor (answer-sdr-question)

File: skills/answer-sdr-question/scripts/answer_sdr_question.py

What it does: answers a free-form SDR question using **only** citation-numbered passages from the knowledge base.

Flow:

1. Uses the **ChromaDB semantic** retriever (top 5 chunks) — not BM25 — because questions are free-form human language. The query is the raw question.
2. The prompt’s key rules: “*Answer using ONLY the numbered sources. Cite sources inline with [n].*”; keep under 200 words; the **next_step** must name **who** to message, **which** channel, and **what** to send (generic “ask your manager” fails); any specific identifier (a TRAIL-XXX-NNN module, an LMG-NNNN policy code, a name, a dollar figure) **must appear verbatim in a source or be omitted**; if the question is too broad, refuse and propose 2–3 narrower questions plus a 1:1.
3. Returns TutorAnswer: **answer**, **next_step**, **confidence**.
4. **Review triggers:** **best_score** = max(retrieval similarities); **flagged** = **best_score** < 0.40 (the semantic weak threshold); **review_required** = **flagged** OR **confidence** < 0.60.
5. Writes no artifact; **next_action** = “done”.

When the manager **approves** a tutor answer, the question/answer pair is written back into the ChromaDB index (add_approved_qa) so future retrieval can reuse human-validated content. A **reject** logs a “gap” the team should fill.

4.6 Cold-Call Simulation (simulate-cold-call + debrief-cold-call)

Files: skills/simulate-cold-call/... and skills/debrief-cold-call/...

What it does: the simulator role-plays a prospect persona one turn at a time; the debrief skill scores the finished conversation.

- **simulate-cold-call** loads a persona from data/raw/sim_personas.json and replies in character (“*You are the prospect, not a coach... never break the fourth wall*”). It pushes back with an objection if the trainee is vague, warms up if discovery is good. It is **stateless** — the UI posts the whole conversation each turn — and has **no review gate**.
- **debrief-cold-call** scores the conversation on the same four weighted rubric dimensions as Coaching Notes (see Section 8.3), writes 01_simulation_debrief.md, and is **advisory** (the trainer coaches from it but it isn’t gated).

4.7 Coaching Notes (score-cold-call)

File: skills/score-cold-call/scripts/score_cold_call.py

What it does: turns a **real** pasted call transcript into a structured, scored coaching note that the manager reviews before it reaches the SDR.

Flow:

1. BM25 retrieval (top 3) of the cold-call rubric.
2. The LLM scores four dimensions 1–10 with rationale, lists what worked / what to improve, and gives **exactly two** “replace X with Y” language alternatives. Tone adapts to the SDR’s experience level.
3. **The server re-computes the weighted total itself** from the dimension scores (it does not trust the LLM’s arithmetic — see Section 8.3).
4. **Review triggers** (any of):
 - confidence < 0.70, or
 - recomputed weighted total < 4.0 (below-bar performance), or
 - any single dimension < 3, or
 - the LLM’s self-reported total differs from the server recompute by more than 1.0 (drift).
5. Writes 04_coaching_note.md; the manager must approve or edit before the SDR sees it.

4.8 Certification Gap Analysis (score-certification)

File: `skills/score-certification/scripts/score_certification.py`

What it does: at the week-12 gate, scores the SDR’s simulation transcripts and certification exam answers against a **5-dimension rubric**, quotes the evidence behind each score, and recommends PASS / BORDERLINE / FAIL — but the trainer makes the real call.

The 5 rubric dimensions (`certification_rubric.md`):

1. Product knowledge accuracy
2. Objection handling (Defuse, Discover, Deliver)
3. MEDDIC framework application
4. Salesforce value messaging
5. Discovery and questioning

Flow:

1. BM25 retrieval (top 5) of the rubric, injected as “the single source of truth.”
2. The LLM scores each dimension 1–10 and, crucially, returns an `evidence_quote` — a **verbatim quote from the SDR’s own artifact** (exam or transcript) — plus a gap-to-close and a coaching action.
3. **The server re-bands every score itself** (the LLM’s self-reported verdict is overwritten — see Section 8.4). This is the anti-“verdict-drift” safeguard.
4. **Review triggers:** server-computed overall is not PASS, **or** confidence < 0.65, **or** the LLM’s overall band disagreed with the server (the disagreement is written into `review_reasons` so the trainer is told *why*).
5. The trainer sees per-dimension cards (score + quoted evidence + gap + action) and makes a **Pass/Fail decision with a required comment**. The AI recommendation is explicitly advisory.

The 7-SDR synthetic cohort the demo uses (4 PASS, 2 BORDERLINE, 1 FAIL) is produced idempotently by `skills/score-certification/scripts/seed_synthetic_cohort.py`, which writes per-SDR transcripts, an exam, optional coaching notes, and a cached gap analysis.

4.9 Onboarding Debrief (generate-handoff-doc)

File: `skills/generate-handoff-doc/scripts/generate_handoff_doc.py`

What it does: *after* an SDR passes certification, they answer **7 process-feedback questions** about their ramp, and the AI synthesises a **manager- and enablement-facing debrief** of what worked and what to fix in the onboarding program. (This is program feedback — not a resignation or account handoff.)

Flow:

1. BM25 retrieval (top 4) for program/ramp context.
2. The prompt: “*You are a Salesforce enablement specialist synthesising a post-certification onboarding debrief... read by the SDR’s Manager and the enablement team... it is NOT a customer/account handoff.*”
3. Returns DebriefDoc: a `summary`, `elements[1-6]` (per onboarding component, each with `what_worked` + `what_to_improve`), `ramp_barriers`, `program_strengths`, `suggested_improvements`, `confidence`. The model is told to lower confidence if the answers were sparse or contradictory.
4. **Review trigger:** the single condition `confidence < 0.65`.
5. Writes `01_handoff_doc.md`; it lands in the Manager’s “Debrief reviews” queue for approve / edit / reject before being shared with enablement.

(Internal names like the folder `generate-handoff-doc`, the `/offboarding/*` routes, and `01_handoff_doc.md` were intentionally kept stable to reduce risk near the deadline; only the behaviour, questions, output shape, and labels were reframed.)

5 How every score and threshold is calculated

This section pulls every number into one place.

5.1 RAG retrieval — two retrievers, two different “weak” rules

Both retrievers run over the same corpus and return the same shape (`{doc_name, passage, score, weak}`) so the UI renders them identically, but they compute scores differently.

BM25 keyword retriever (`rag/retrieval.py`) — used by the chain skills, coaching, and certification:

- Library: `rank-bm25` (BM25Okapi). Deterministic, offline, no API key.
- Docs are split into **paragraph chunks** (split on blank lines after stripping YAML frontmatter); the index is built recursively over `rag/knowledge_base/`.
- The score is the **raw, unbounded BM25 score** — there is no fixed cutoff. Instead the “weak” rule is **relative**: a source is weak when its score is **below 30% of the top score** in that same retrieval (`WEAK_RATIO_DEFAULT = 0.30, retrieval.py:138`).
- Default **top-k = 3**.

ChromaDB semantic retriever (`rag/embeddings_retrieval.py`) — used **only** by the AI Tutor:

- Embedding model: ChromaDB’s default `all-MiniLM-L6-v2` (Sentence-Transformers, runs locally). Cosine space.
- Docs are split into **300-word windows with 40-word overlap**.
- ChromaDB returns a cosine **distance**; the code converts it to a bounded **[0,1] similarity**: `similarity = max(0.0, 1.0 - distance)` (`embeddings_retrieval.py:205`).
- Because the scale is bounded, the “weak” rule here is an **absolute** cutoff: a source is weak when `similarity < 0.40` (`LOW_CONFIDENCE_THRESHOLD = 0.40, line 39`). This is the same number the Tutor uses to set `flagged`.
- Default **top-k = 5**.
- **Write-back loop**: approved tutor Q&A pairs are upserted into the same collection (tagged `source="approved_qa"`), so human-validated answers re-enter retrieval.

Knowledge base size: 16 internal “proprietary” docs (policy codes, ramp percentages, dollar bands, the certification rubric + exam bank, etc.) + 7 scraped Wikipedia reference docs = **23 docs total**.

5.2 Confidence thresholds — the complete table

Every gate in the system reduces to one number. Here they all are:

Step / skill	Gate fires when...
Generate onboarding plan	confidence < 0.70 , or off-allow-list module, or missing summary
AI Tutor	retrieval best score < 0.40 , or confidence < 0.60
Coaching note (score-cold-call)	confidence < 0.70 , or total < 4.0 , or any dim < 3 , or drift > 1.0
Simulation debrief	same four as coaching (advisory, not gated)
Certification	overall PASS, or confidence < 0.65 , or LLM band server band
Onboarding debrief	confidence < 0.65
Pre-boarding chain default	confidence < 0.60 (<code>needs_human_review</code> default)

5.3 Cold-call / coaching score — the weighted average

The four rubric dimensions (Coaching Rubric v3.1, `cold_call_quality_rubric.md`) are weighted and summed by the **server**, not the model:

$$\text{total} = 0.20 (\text{opening}) + 0.35 (\text{discovery}) + 0.25 (\text{objection}) + 0.20 (\text{close})$$

Each dimension is 1–10; the weights sum to 1.0, so the total is also on a 1–10 scale. The server keeps **both** its own recomputed total (`weighted_total`) and the LLM’s self-reported one (`llm_weighted_total`) and flags the

case if they differ by more than 1.0. The rubric also defines deductions (e.g. pitch-dumping -2 , an unapproved competitor claim -3 *and* escalate) that guide the dimension scores.

5.4 Certification — banding, not averaging

Certification does **not** average the five scores. The server bands them with a **worst-dimension rule** (`compute_overall`, lines 80–92):

- **any** dimension $4 \rightarrow$ **FAIL**
- else **any** dimension is 5 or 6 $\rightarrow$ **BORDERLINE**
- else (all 7) $\rightarrow$ **PASS**

The same thresholds re-band each individual dimension. This server-side recompute **overrides** whatever band the LLM reported; if they disagree, the mismatch is appended to `review_reasons`. The result is a **category**, not a number — the weakest dimension is reported separately.

5.5 Mentor ranking — see Section 6.2

The deterministic 5-component 0–100 mentor score is fully specified in the Step-2 table above.

6 What we are doing better, and how the numbers are derived

All time/cost figures are **reasoned assumptions with labelled ranges**, anchored to the Milestone-01 baseline — they are not measured company data. The honesty of that labelling is itself part of the deliverable.

6.1 Time saved per step

Step	Current	AI-supported	Why it changes
Pre-boarding plan	90–150 min	10–20 min	AI drafts from template + RAG; human spot-checks
Rank a mentor	20–40 min	5–10 min	deterministic ranked table replaces the search
Welcome email	15–30 min	3–8 min	template slot-fill
Answer a question	10–30 min	1–3 min	tutor retrieves + cites in seconds
Coaching note	20–40 min	5–12 min	AI drafts from transcript; manager reviews
Certification	45–75 min	10–15 min	AI scores the rubric + quotes evidence
Post-cert debrief	60–90 min (rarely done)	10–15 min	AI synthesises; manager reviews

6.2 How the cost numbers are calculated

The formula is deliberately simple and stated in `estimates.md`:

$$\text{labor cost per case} = \frac{\text{minutes per case}}{60} \times \$70/\text{hr}$$

\$70/hr is a labelled assumption for a fully-loaded US enablement/manager role. Applied to the **pre-boarding cycle** (the first three steps summed):

- Current: 125–220 min → **\$146–\$257 per hire**
- AI-supported: 18–38 min → **\$21–\$44 per hire**
- **Saving \$120–\$215 per hire**

Certification adds a one-time **\$40–\$71 per SDR** saving (45–75 → 10–15 min). The largest *recurring* saving is coaching, which moves a manager from the Milestone-01 baseline of 3–5 hrs/week of call review toward the under-1-hr/week target.

6.3 The four quality improvements (not just time)

1. **Better use of internal documents** — both LLM skills retrieve from the 16-doc KB and cite their sources; the tutor refuses when no source exists.
2. **More consistent coaching** — the same rubric every time, instead of ad-hoc verbal feedback (Milestone-01’s “largest uncontrolled variable”).
3. **Faster feedback** — drafting from a transcript in minutes moves latency from 24–72 hrs toward under 4 hrs.
4. **Auditability** — every decision is logged; certification adds a server-side band recompute that catches LLM drift before it reaches the trainer.

7 How we tested it

There are **two separate test layers**, kept apart by pytest markers in `pyproject.toml` (the LLM-hitting suites are deselected by default).

7.1 Layer 1 — the offline pytest suite (153 tests)

- **153 tests pass with no API key and no network.** They run offline because each skill honours `MOCK_LLM_RESPONSE`: the test sets that env var to a realistic JSON payload, and the skill validates it straight into its schema instead of calling the model — so the full pipeline (schema, score recompute, review logic, artifact writing, envelope contract) is exercised for real.
- Rough distribution: `tests/utils` 48, `tests/skills` 54, `tests/rag` 23, `tests/mcp_servers` 14, `tests/automations` 13, plus the orchestrator test.
- Example coverage: the certification band logic (7 PASS / 5–6 BORDERLINE / 4 FAIL), the LLM-vs-server drift path, the low-confidence review trigger, the BM25 weak-flag rule (score < 30% of top), and the 7-key envelope contract for every skill.

7.2 Layer 2 — the DeepEval quality suite (the milestone’s testing deliverable)

This suite measures the **AI Tutor’s answer quality** against the real LLM. The scope phrase to remember: **10 test cases × 4 checks = 40 metric results per run**. (“40” is metric *results*, not 40 tests.)

The 10 cases (`tests/eval_suite/dataset.py`) span four categories — 7 `normal`, 1 `vague`, 1 `no_source`, 1 `confident_incomplete` — e.g. handling a price objection, the cold-call rubric, the quota ramp, territories, a silent-enterprise escalation, first-week tooling, competitor-pricing legality, “*how can I get better at my job?*” (the deliberately vague one), a fabricated-ROI-policy question (no source exists), and “*walk me through a perfect cold call*” (needs two docs, so citing one is incomplete).

The 4 checks (each graded at a **0.70 pass threshold**, three of them scored on a strict 0.0 / 0.5 / 1.0 scale, judge model `claude-sonnet-4-6`):

Check	Type	What it verifies
CitationAndNextStep	deterministic (regex)	answer contains a [n] citation and a real next step
Faithfulness	LLM-judged	every claim is supported by the retrieved passages (or the tutor declines)

Check	Type	What it verifies
AnswerRelevancy	LLM-judged	the answer addresses the question against a per-case rubric
SDRFaceValidity	LLM-judged	would a new SDR find it concrete, peer-toned, actionable

A case is marked PASS only if **all four** pass — deliberately strict, so a merely-okay answer (a 0.5) fails.

`test_tutor_quality.py` runs the **real** tutor skill + **real** semantic retrieval, grades with the four metrics, and writes a timestamped report to `tests/eval_suite/results/`.

7.3 The before/after experiment (we changed one thing and measured)

We ran the suite **six times** — **three before a prompt change, three after** — to separate real signal from LLM-judge noise. The one change added three rules to the tutor’s system prompt: (1) the `next_step` must be a concrete verb-noun action (who/channel/what), (2) block any ungrounded identifier, (3) refuse over-broad questions and propose narrower ones.

	Run 1	Run 2	Run 3	Mean	Range
Before	23	20	21	21.3 / 40	3
After	24	23	23	23.3 / 40	1

Two real results: the **mean rose +2**, and — just as importantly — the **variance collapsed from range 3 to range 1** (every post-fix run now agrees). The cleanest single signal: the vague-question case (`vague_get_better`) went from passing AnswerRelevancy **1 of 3 runs to 3 of 3** once the broad-question rule was added.

8 Failures and limitations we actually found

We document **eight predicted failure cases** (`failure_cases.md`); four are exercised directly by the DeepEval suite. The honest findings:

1. **Dominant remaining failure — KB-vs-rubric mismatch.** On several cases the tutor gives a faithful, cited answer that still misses part of what the case rubric wanted, *because the KB doc doesn’t contain that facet*. This is a content-alignment problem, not a prompt problem, and is the biggest open item.
2. **Partly-fixed fabrication.** The “block ungrounded identifiers” rule eliminated the fabricated TRAIL-COLD-101 module ID (case 10 Faithfulness went 2/3 → 3/3), but a *stylistic* fabrication on the price-objection case (“the goal is a meeting, not a negotiation”) is a phrase, not an identifier, so the rule only catches it 1 of 3 runs.
3. **Refusals score badly on relevancy.** When the right answer is “I can’t answer this,” the relevancy judge still wants the substantive answer its rubric expects, so a correct refusal can score 0.00. The fix is to give `no_source` cases a rubric that rewards refusal.
4. **A genuine regression, disclosed.** The same prompt change that helped four cases *hurt* two on SDRFaceValidity (cases 3 and 9) — the more-prescriptive prompt made two answers read like internal documentation rather than peer advice. The face-validity headline is therefore flat, which we report rather than hide. The prompt change is a tradeoff, not a free lunch.

Honest caveats on the estimates (`face_validity.md`):

- The **post-cert debrief has no Milestone-01 baseline** (structured ramp feedback was rarely captured before), so its time estimate is the least-anchored figure.
- The **recurring coaching saving** depends on an assumed call volume per rep per week — the per-call saving is grounded, the weekly total is not measured.

- **Certification is demoed on a synthetic 7-SDR cohort**, not real SDR artifacts; the data model accepts real artifacts but the live wiring is deferred.

All three are flagged as low-confidence rather than presented as settled.

9 The safety net: humans stay in control

Five explicit review gates remain (`human_review_plan.md`):

1. **Manager gate** at the end of the pre-boarding chain.
2. **SDR/Manager approve-or-reject** on every tutor answer (a reject is logged as a knowledge gap).
3. **Manager review** of every coaching note before it reaches the SDR.
4. **Trainer Pass/Fail** on certification — required comment, AI is advisory, the server-recomputed band is the source of truth.
5. **Manager review** of every onboarding debrief before it is shared.

The four controls — **approve** (proceed), **edit** (store the corrected version, then proceed), **reject** (stop, rerun), **escalate** (route to a human owner) — are each appended to `data/decision_log.json` with the feature, action, context, and timestamp. The dashboard reads this log to show approve/edit/reject/escalate rates per feature. The three consequential Milestone-01 checkpoints (certification, territory assignment, qualified-meeting verification) stay fully human.

10 One-line summary

A new SDR gets a personalised plan, a matched mentor, a cited tutor, realistic practice, consistent coaching, an evidence-backed certification, and a debrief that improves the program for the next cohort — produced in **18–38 minutes** of manager time per pre-boarding cycle instead of 125–220, with **every AI output checked by a human at five gates and every decision logged**.